GOVERNANCE · MODEL MONITORING

Drift report

Has tonight's book moved away from what the model learned on?

Team Sharpe Shooters · CIBC Collections Hackathon · 9 October 2026

8 · 5 · 13
drift · watch · stable, of 26 features

690K
training rows (Sep 2025 – Sep 2026)

96K
cases scored tonight

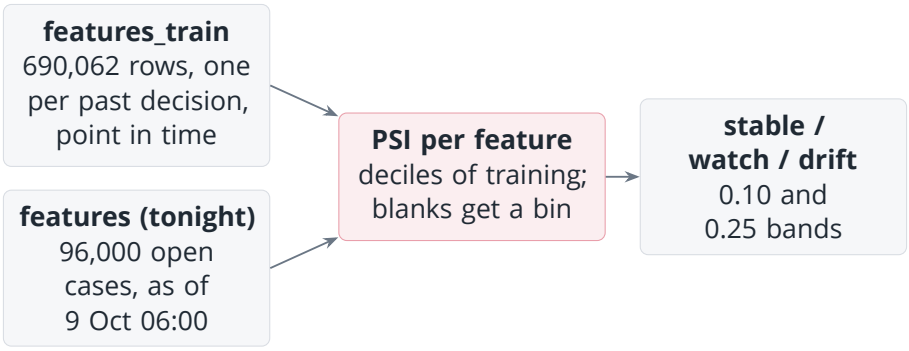
0% → 22%
training rows with payroll delay and pay change

The short version. Eight features cross the drift line; none of them is a reason to stop scoring.

- **Fixed since the last report:** payroll delay and pay change were blank in every training row. They now come from the dated monthly salary feed, with one definition for training and live, and are filled about equally in both (PSI 0.02 and 0.01). We retrained with them, and the champion/challenger gate kept the current model: pay shocks don't add uplift signal beyond noise on this data. The payday effect lives in the promise's due date (evidence report §2), not in who to contact.
- **The loud but mostly mechanical ones:** cohort rates (PSI 5.5 and 4.0) and the delinquency features. Tonight is one snapshot of the open book, while training is a year of decision points. Within each cohort the kept rate is only 1–3 points below where training ended.
- **A silent one:** `promise_intent` is a model input that is blank in both training and live. Its PSI of 0 means "nothing to compare", not "stable". It leaves the inputs at the next promoted retrain.
- **An artefact of the date:** `contacts_last_7d` (4.45) is zero for every case tonight, because the release's contact history ends on 28 September and tonight is 9 October. Not a model input; in production the feed is daily.

1 How to read PSI

PSI (population stability index) compares two distributions bin by bin: $\sum(live\% - train\%) \times \ln(live\%/train\%)$. Numbers are cut at the training deciles, categories by value, and **missing values get their own bin**, so a feature that suddenly fills in or goes blank shows up too. We use the usual bank bands: **below 0.10 stable**, **0.10–0.25 watch**, **0.25 or more drift** (investigate, then decide on a retrain).



2 Which features moved

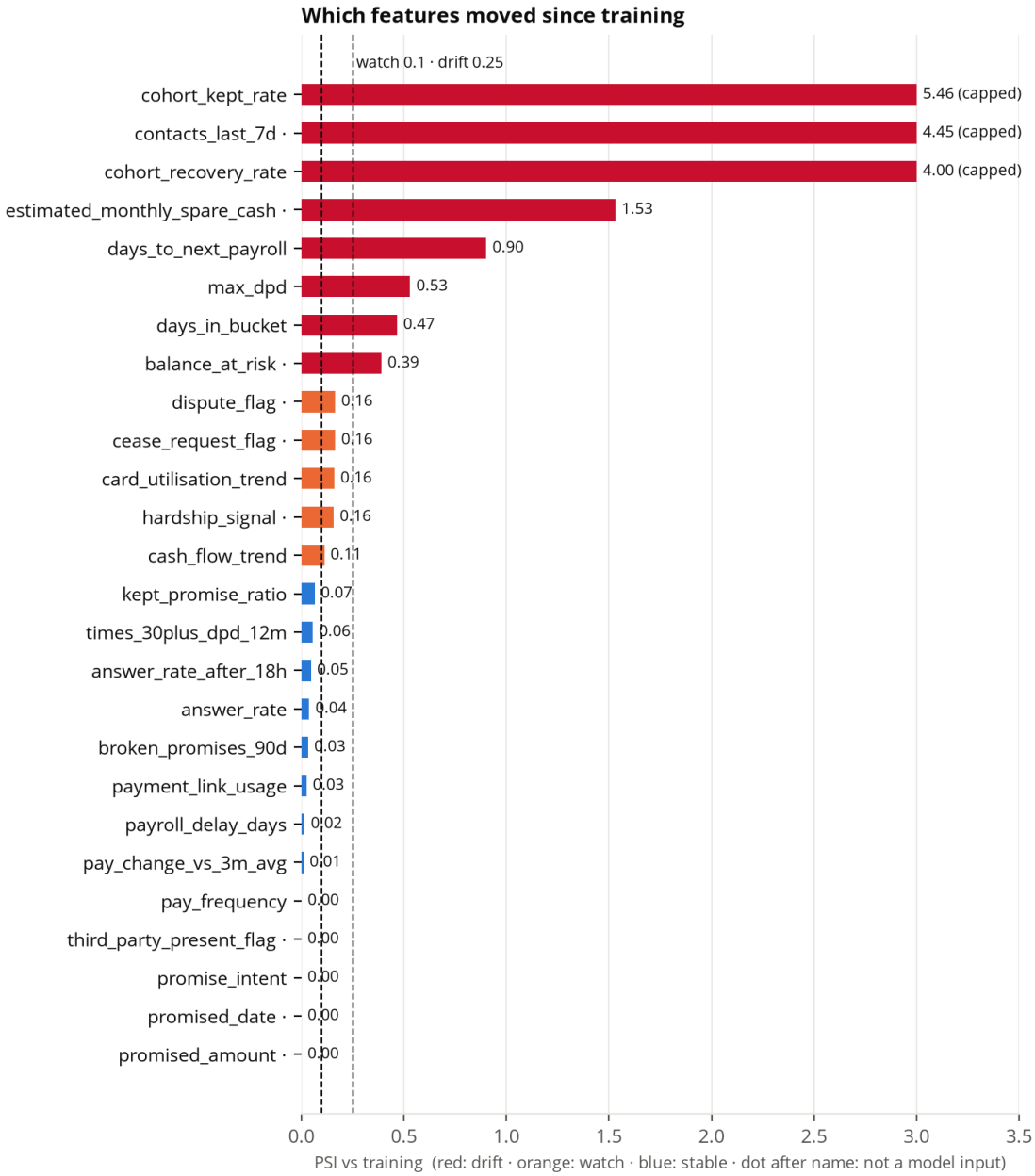


Figure 1. PSI of every registered feature, training vs tonight. Red is drift, orange is watch, blue is stable.

What the chart says. The red bars fall into three groups, each with a different story.

Cohort rates: loud, mostly mechanical. `cohort_kept_rate` (5.46) and `cohort_recovery_rate` (4.00) top the chart. Each is a cumulative rate per product × entry bucket, recomputed at every point in time. In training that gives 3,229 distinct values drifting slowly through the year. Tonight there is one as-of date, so there are only 6 values, one per cohort, and each lands in one or two training deciles. PSI punishes that heavily. The real movement is small:

Cohort (entry 1–30 days)	Last training value	Tonight	Training rows	Cases tonight
Card	0.587	0.569	311,758	38,320
Personal loan	0.478	0.450	143,749	23,960
Auto loan	0.483	0.453	88,291	15,340
Line of credit	0.524	0.497	66,003	10,540
Overdraft	0.604	0.587	62,472	6,860
Mortgage	0.646	0.640	17,789	980

Table 1. Promise-kept rate per cohort: the last value the model trained on vs tonight's.

The direction is consistent, though. Across the training year the median rate slid from 0.64 (Q3 2025 to Q2 2026) to 0.59 (Q3 2026), and every cohort is a little lower again tonight. Promises are being kept slightly less often than a year ago. That's worth watching, but it's not a reason to stop scoring.

Payroll features: fixed. `payroll_delay_days` and `pay_change_vs_3m_avg` used to come from the deposit snapshot, which is dated after every training decision, so training left them blank. They now come from the dated monthly salary feed (`salary_credit_history`, employer pay only), read as of the earlier of the as-of date and the feed's last complete month (`dbt/models/features/account_features.sql`). The same definition runs for training and live, so they're filled about equally (22% and 24%) and stable. Retrained with them, the challenger lost to the champion beyond noise, so the gate kept the champion. `estimated_monthly_spare_cash` still needs the snapshot's spend fields: live only.

Delinquency features: a different slice, as expected. `max_dpd` (0.53), `balance_at_risk` (0.39) and `days_in_bucket` (0.47) moved because tonight is the open book, not a stream of decisions. The median open case is 52 days past due, against 29 at a training decision, and has $437 at risk against $135. `contacts_last_7d` (4.45) is an artefact of the date: the release's contacts end on 28 September, so no case has one in the 7 days before 9 October.

3 What went blank, or filled in

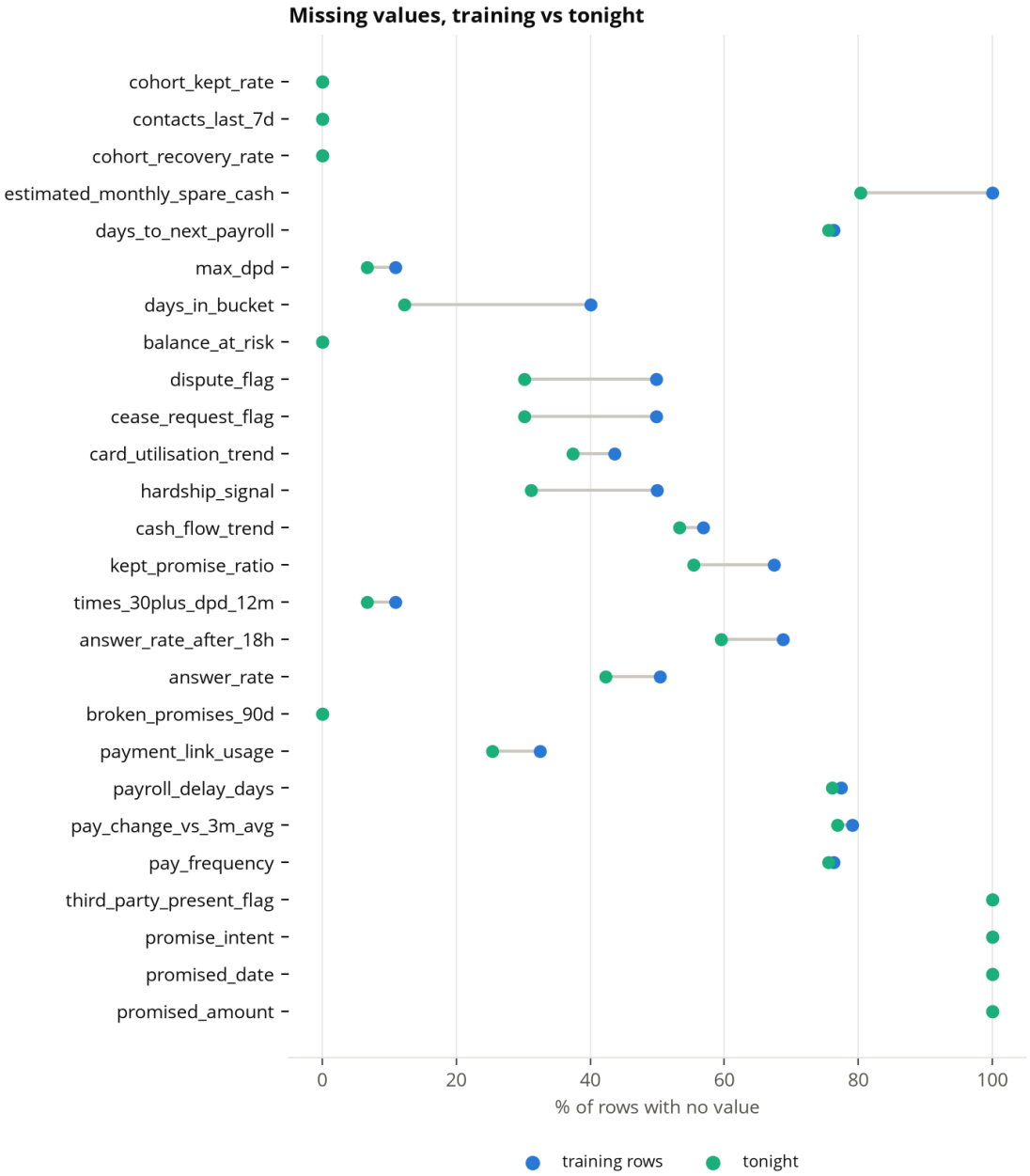


Figure 2. Share of rows with no value: training (blue) vs tonight (green).

What the chart says. Most lines run right to left: features are *more* complete tonight than in training. Payroll delay and pay change are now blank about as often in training (77–79%) as tonight (76–77%); spare cash goes from 100% to 80%. The text flags (`hardship_signal`, `dispute_flag`, `cease_request_flag`) go from 50% to 30% blank: a blank here means no note on file, and more of tonight's open cases have a note than past decisions did. That fill-in is why those three are in "watch" at 0.16–0.17, not because customers changed. The four dots stuck at 100% on both sides (`promise_intent`, `promised_date`, `promised_amount`, `third_party_present_flag`) are the silent problem. Their PSI is 0 because there's nothing to compare. `promise_intent` is a model input that has never had a value.

4 Every feature

Feature	PSI	Status	Blank, training	Blank, tonight	Model input
cohort_kept_rate	5.46	drift	0%	0%	yes
contacts_last_7d	4.45	drift	0%	0%	
cohort_recovery_rate	4.00	drift	0%	0%	yes
estimated_monthly_spare_cash	1.53	drift	100%	80%	
days_to_next_payroll	0.90	drift	76%	76%	yes
max_dpd	0.53	drift	11%	7%	yes
days_in_bucket	0.47	drift	40%	12%	yes
balance_at_risk	0.39	drift	0%	0%	
dispute_flag	0.16	watch	50%	30%	
cease_request_flag	0.16	watch	50%	30%	
card_utilisation_trend	0.16	watch	44%	37%	yes
hardship_signal	0.16	watch	50%	31%	
cash_flow_trend	0.11	watch	57%	53%	yes
kept_promise_ratio	0.07	stable	67%	55%	yes
times_30plus_dpd_12m	0.06	stable	11%	7%	yes
answer_rate_after_18h	0.05	stable	69%	60%	yes
answer_rate	0.04	stable	50%	42%	yes
broken_promises_90d	0.03	stable	0%	0%	yes
payment_link_usage	0.03	stable	32%	25%	yes
payroll_delay_days	0.02	stable	77%	76%	yes
pay_change_vs_3m_avg	0.01	stable	79%	77%	yes
pay_frequency	0.00	stable	76%	76%	yes
third_party_present_flag	0.00	empty	100%	100%	
promise_intent	0.00	empty	100%	100%	yes
promised_date	0.00	empty	100%	100%	
promised_amount	0.00	empty	100%	100%	

Table 2. All 26 registered features. “Empty” rows have nothing to compare, so a PSI of 0 there means nothing.

5 What we do with this

1. **Payroll in training: done.** Delay and pay change come from the dated salary feed; the retrain didn’t beat the champion, so it wasn’t promoted. Spare cash needs dated spend: keep a copy of the deposit snapshot each night, then add it.
2. **Teach the check about cohorts.** Compare cohort rates to the last training value per cohort, with a tolerance of a few points, instead of using decile PSI. Today’s check cries wolf on them every night.
3. **Fill or drop the empty inputs.** The SetFit backfill only predicts hardship, so the promise fields can only come from the LLM extraction, and 0 of the 67,039 rows in `features.text_features` carry one. `promise_intent` leaves the model’s inputs at the next promoted retrain (the live champion was trained with it, blank). A feature that’s always blank is just noise in the registry.
4. **Keep the nightly gate.** `kept drift` runs after `kept features`. A drifted *model input* that

none of the reasons above explains holds the new scores and pages the owner.

Not a fairness check

Drift asks whether the population moved; the fairness report asks whether groups are treated differently. Different questions with different reports, and both run nightly.

Numbers `scripts/drift_report.py` compares `features_train.case_features` with `data/exports/features/case_features.parquet` and caches the result in `data/exports/reports/drift_data.json`; the per-cohort table is a one-off query on the same tables.

Rebuild `uv run python scripts/drift_report.py` (with `kept serve` stopped), then `uv run python scripts/reportlib.py drift_report`